

교과목 3 : 초거대언어모델(LLM)

## 단원 2 : 자연어 딥러닝

DAY1. 자연어 처리를 위한 딥러닝 모델

# 목 차

|                   |       |    |
|-------------------|-------|----|
| 1. 자연어 처리 딥러닝 모델  | ..... | 3  |
| 2. 자연어 처리를 위한 CNN | ..... | 17 |
| 3. 자연어 처리를 위한 RNN | ..... | 24 |

# 1. 자연어 처리 딥러닝 모델

자연어 처리(NLP, Natural Language Processing)에서 딥러닝 모델은 사람이 사용하는 언어(한국어, 영어 등)를 컴퓨터가 이해하고 처리하도록 학습하는 인공지능 모델이다. 과거에는 사람이 직접 규칙을 만들어 프로그램을 작성했지만, 현재는 대부분 딥러닝 모델이 대량의 문장을 학습하여 스스로 언어의 특징을 배우는 방식이 사용된다.

## 1. 자연어 처리란?

자연어 처리(NLP)는 사람이 사용하는 언어를 컴퓨터가 이해하고 분석하며 생성하는 기술이다.

예를 들면 다음과 같다.

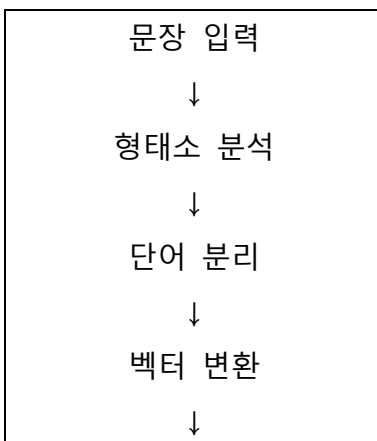
- 문장 분류
- 감성 분석
- 기계 번역
- 문서 요약
- 질의응답
- 챗봇
- 음성 인식
- 문장 생성

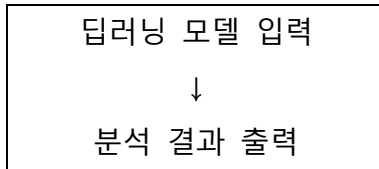
예시

입력

오늘 날씨가 정말 좋네요.

컴퓨터의 처리 과정





## 2. 자연어 처리에서 딥러닝이 필요한 이유

사람의 언어는 매우 복잡하다.

예를 들어

배가 아프다  
와  
배를 먹었다

의 "배"는 의미가 다르다.

첫 번째  
배 = 신체  
두 번째  
배 = 과일

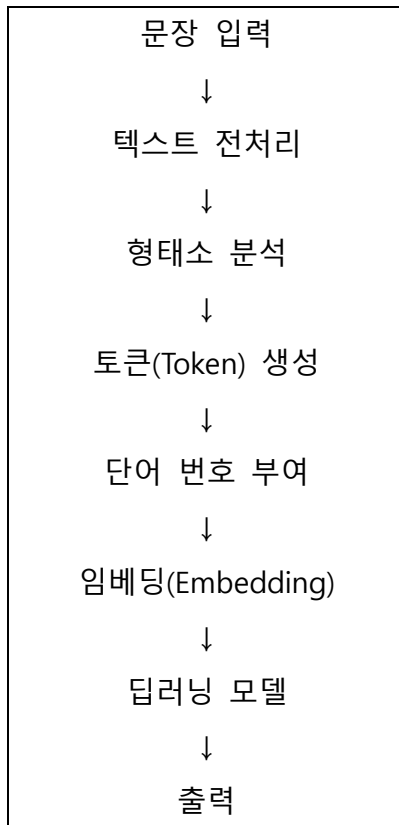
컴퓨터는 이런 의미를 스스로 판단하기 어렵다.

딥러닝 모델은

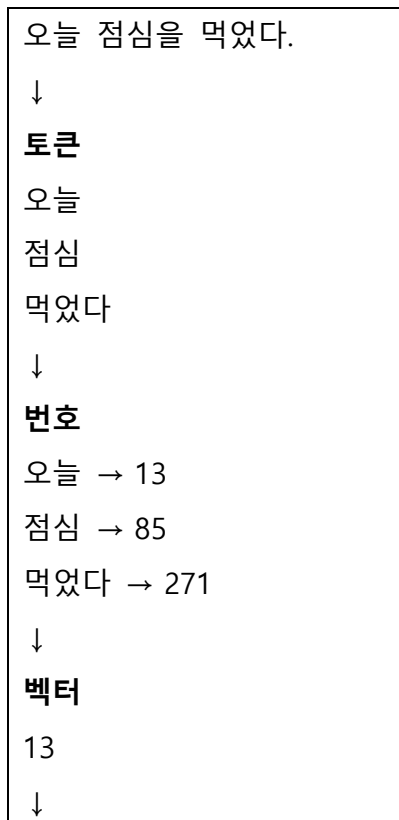
- 문맥(Context)
- 주변 단어
- 문장의 구조

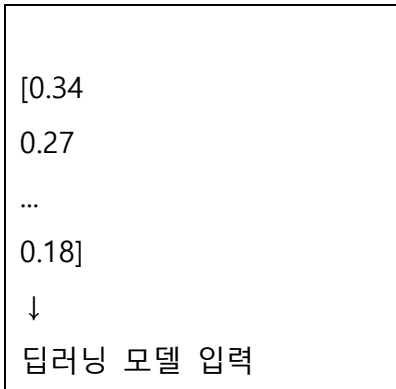
를 함께 학습하여 의미를 이해한다.

### 3. 자연어 처리 딥러닝의 전체 과정



예시





## 4. 자연어 처리에서 사용하는 대표적인 딥러닝 모델

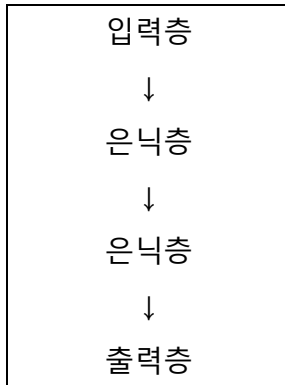
현재 많이 사용하는 모델은 다음과 같다.

| 모델          | 특징         |
|-------------|------------|
| DNN         | 기본 신경망     |
| CNN         | 지역 특징 추출   |
| RNN         | 순차 데이터 처리  |
| LSTM        | 긴 문장 기억 가능 |
| GRU         | LSTM 을 단순화 |
| Seq2Seq     | 번역 모델      |
| Attention   | 중요 단어 집중   |
| Transformer | 병렬 처리 가능   |
| BERT        | 문장 이해      |
| GPT         | 문장 생성      |

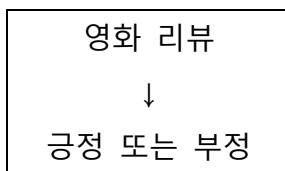
## 5. DNN(Deep Neural Network)

가장 기본적인 딥러닝 모델이다.

구조



예시



장점

- 구조가 간단
- 구현이 쉬움

단점

문장의 순서를 모른다.

예

나는 밥을 먹었다.  
와  
먹었다 나는 밥을.

를 거의 동일하게 본다.

## 6. CNN(Convolutional Neural Network)

원래는 이미지 분석용이다. 하지만 자연어 처리에도 많이 사용된다.  
CNN 은 연속된 단어 패턴을 잘 찾는다.

예

정말 재미있는 영화

여기서

재미있는 영화

를 하나의 특징으로 학습한다.

사용 분야

- 감성 분석
- 문장 분류
- 스팸 메일

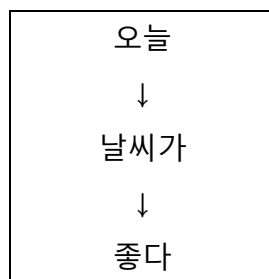
장점

속도가 매우 빠르다.

## 7. RNN(Recurrent Neural Network)

RNN 은 문장의 순서를 기억한다.

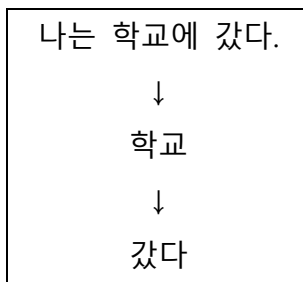
구조



앞에서 본 정보를 다음 단어에 전달한다.



예



앞의 정보를 기억한다.

하지만 긴 문장을 잘 기억하지 못한다.

## 8. LSTM(Long Short-Term Memory)

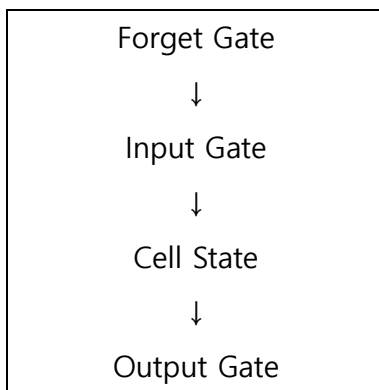
LSTM 은 RNN 의 단점을 해결하였다.

핵심

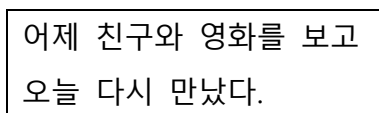
필요한 정보만 기억

불필요한 정보는 삭제

구조



예



"친구" 정보를 끝까지 기억한다.

장점

- 긴 문장 처리 가능
- 번역
- 음성 인식

## 9. GRU(Gated Recurrent Unit)

GRU 는 LSTM 을 단순화하였다.

LSTM

게이트 3 개

GRU

게이트 2 개

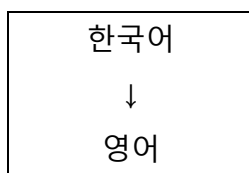
장점

- 속도 빠름
- 메모리 적게 사용
- 성능은 거의 비슷

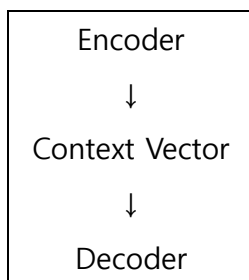
## 10. Seq2Seq(Sequence to Sequence)

문장을 다른 문장으로 바꾸는 모델이다.

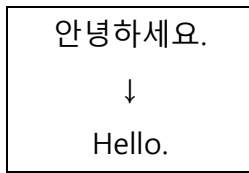
예



구조



예



사용

- 번역
- 요약
- 챗봇

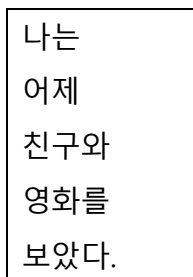
## 11. Attention

Seq2Seq 의 단점을 해결하였다.

문장이 길어지면 중간 벡터 하나만으로는 부족하다.

Attention 은 필요한 단어를 다시 본다.

예



"영화"를 번역할 때 "영화" 단어에 집중한다.

장점

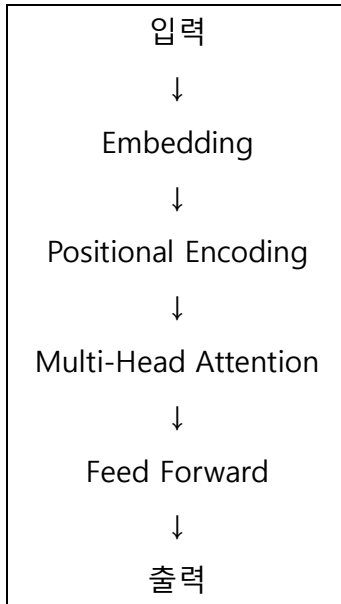
긴 문장에서도 성능 향상

## 12. Transformer

현재 자연어 처리의 핵심 모델이다.

Transformer 는 RNN 을 사용하지 않는다. 대신 Attention 만 사용한다.

구조



장점

- 매우 빠름
- 병렬 처리 가능
- 긴 문장 처리 가능

현재 대부분의 최신 NLP 모델은 Transformer 구조를 기반으로 한다.

### 13. BERT(Bidirectional Encoder Representations from Transformers)

BERT 는 Transformer 의 **인코더(Encoder)** 구조를 사용하는 모델이다.

특징

양쪽 방향을 동시에 읽는다.

예

은행에 갔다.

앞뒤 문맥을 모두 참고하여

은행

의 의미를 이해한다.

활용 분야

- 문장 분류
- 질의응답
- 개체명 인식
- 검색

장점

문장의 의미를 매우 잘 이해한다.

### 14. GPT(Generative Pre-trained Transformer)

GPT 는 Transformer 의 **디코더(Decoder)** 구조를 사용하는 생성형 언어 모델이다.

동작

다음 단어를 예측

예

### 입력

오늘은 날씨가

### 예측

좋다

↓

다음

오늘은 날씨가 좋다

↓

다음

오늘은      날씨가      좋다.

그래서

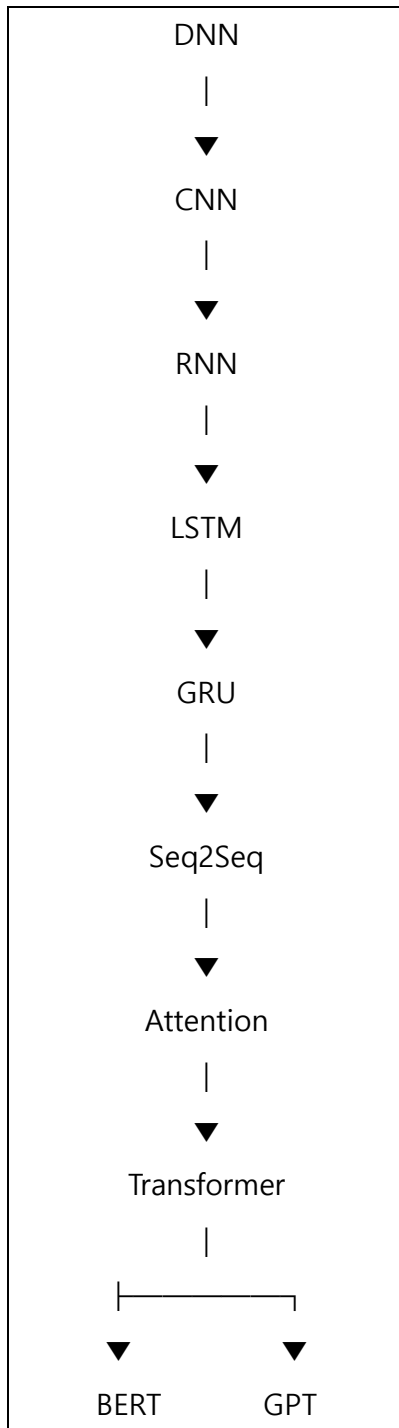
↓

계속 생성

### 활용

- 챗봇
- 문장 생성
- 코드 생성
- 번역
- 요약

## 15. 모델 발전 과정



## 16. 모델별 특징 비교

| 모델          | 순서 정보       | 긴문장 처리 | 병렬 처리 | 주요 활용             |
|-------------|-------------|--------|-------|-------------------|
| DNN         | X           | X      | ○     | 기본 분류             |
| CNN         | 부분적         | △      | ○     | 문장 분류, 감성 분석      |
| RNN         | ○           | X      | X     | 시계열, 간단한 문장       |
| LSTM        | ○           | ○      | X     | 번역, 음성 인식         |
| GRU         | ○           | ○      | X     | 실시간 분석            |
| Seq2Seq     | ○           | ○      | X     | 기계 번역             |
| Attention   | ○           | ○      | 부분적   | 번역, 요약            |
| Transformer | ○(위치 정보 사용) | ○      | ○     | 대부분의 NLP 작업       |
| BERT        | ○           | ○      | ○     | 문장 이해, 검색, 분류     |
| GPT         | ○           | ○      | ○     | 텍스트 생성, 챗봇, 코드 생성 |

## 17. 자연어 처리 모델의 실제 활용 사례

- **감성 분석:** 상품 리뷰나 영화 리뷰를 긍정·부정으로 분류
- **기계 번역:** 한국어를 영어, 일본어 등으로 자동 번역
- **챗봇 및 AI 비서:** 사용자의 질문을 이해하고 자연스러운 답변 생성
- **문서 요약:** 긴 뉴스 기사나 보고서를 핵심 내용만 요약
- **검색 시스템:** 사용자의 검색 의도를 이해하여 적절한 결과 제공
- **음성 비서:** 음성을 텍스트로 변환한 뒤 의미를 분석하여 명령 수행
- **코드 생성 및 보조:** 자연어 설명을 바탕으로 프로그래밍 코드 작성 및 수정 지원

자연어 처리 딥러닝 모델은 언어를 이해하고 생성하는 능력을 지속적으로 발전시켜 왔다. 초기에는 DNN, CNN, RNN 과 같은 비교적 단순한 구조가 사용되었고, 이후 LSTM 과 GRU 가 긴 문장의 문맥을 처리하는 성능을 향상시켰다. Seq2Seq 와 Attention 은 번역과 요약 같은 문장 간 변환 작업의 정확도를 높였으며, Transformer 는 병렬 처리와 문맥 이해 능력을 크게 개선하여 현대 NLP 의 기반이 되었다.

Transformer 를 기반으로 한 BERT 는 문장의 의미를 깊이 이해하는 데 강점을 가지며, GPT 는 자연스러운 문장과 코드를 생성하는 데 뛰어난 성능을 보인다. 현재의 챗봇, 검색 엔진, 번역 서비스, 문서 요약, 코드 생성 등 대부분의 최신 자연어 처리 서비스는 Transformer 계열 모델을 기반으로 동작하고 있으며, 앞으로도 이러한 구조를 중심으로 더욱 발전할 것으로 전망된다.



## 2. 자연어 처리를 위한 CNN

### 합성곱 신경망(CNN)이란?

- 대상에 대한 특징을 인식하는 방식을 모델링
- 이미지 데이터를 다양한 방법으로 필터링하여 학습
- 이미지 학습을 위한 효율적인 알고리즘
- 컴퓨터 비전, 이미지 인식, 음성 인식, 자연어 처리에 활용

### CNN의 기본 연산

- 컨볼루션(Convolution)
- 풀링(Pooling)
- 완전 연결층(Fully Connected Layer)

→ 여러 가지 사물 중 해당 항목에 대한 확률을 출력한다.

#### ① 이미지 특징 추출

- CNN을 기반으로 이미지가 가지고 있는 특징을 추출
- 앞에서 추출한 특징을 이용하여 새로운 이미지도 식별 가능

#### ② 컨볼루션(Convolution)

- 패딩(Padding), 스트라이드(Stride)와 함께 수행
- 추출된 특징을 이용하여 다른 이미지와의 일치 여부 확인

#### ③ 맥스 풀링(Max Pooling)

- 최댓값만 추출하는 과정

#### ④ 플래트닝(Flattening)

- 완전 연결층으로 연결
- 1차원 구조로 변환하여 다시 학습

#### ⑤ 소프트맥스(Softmax)

- 어떤 분류에 속하는지 판단하기 위해 사용

## 컨볼루션 연산 예시

- 4×4 이미지가 2×2 형태로 압축
- 파라미터(Parameter) 공유

### 패딩(Padding)

- 입력 데이터 주변을 특정 값으로 채움
- 출력 크기를 조정하기 위해 사용

### 스트라이드(Stride)

- 이미지 이동 간격 설정

### 스트라이드 공식

입력 이미지 크기 ;  $n \times n$

필터 크기 :  $f$

출력 크기 :  $((n-f)/\text{Stride})+1$

예)

Stride = 1

$((7-3)/1)+1=5$

Stride = 2

$((7-3)/2)+1=3$

Stride = 3

$((7-3)/3)+1=2.33$

## 패딩의 예

- 이미지 정보를 최대한 보존

입력 5×5

필터 3×3

패딩 1

스트라이드 1

출력 5×5

## 풀링(Pooling)

컨볼루션 연산 후 수행

### 최대 풀링(Max Pooling)

- 최대값 선택

### 평균 풀링(Average Pooling)

- 평균값 선택

## 완전 연결층

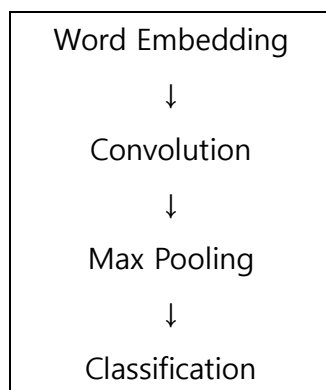
- 압축된 특징을 이용하여 최종 분류
- 다중 분류에는 Softmax 사용
- Softmax Cross Entropy 목적함수 사용

## 컨볼루션 층을 깊게 만드는 이유

- 깊어질수록 더 복잡한 특징 추출 가능
- 고차원의 잠재적인 특징 학습 가능

## CNN을 활용한 문장 분류 모델

### 문장 분류 과정



### 정상 메일과 스팸 메일 분류 실습 과정

- 패키지와 데이터 불러오기
- 불필요한 열 삭제
- 영문 데이터를 숫자로 변경
- 데이터 정보 확인
- 결측값 확인
- 유니크 값 확인
- 중복 데이터 삭제
- `drop_duplicates()` 사용
- 메일 비율 확인
- `value_counts()` 사용
- 정상 메일 비율이 약 80%

### 출력 결과

- `data.info()`
- 결측값 확인
- 중복 여부 확인
- `drop_duplicates()`

## CNN을 이용한 문장 분류

### 데이터 전처리

- train\_test\_split()
- test\_size 지정
- 훈련/평가 데이터 분리
- Tokenizer 생성
- X\_train\_encoded 확인
- tokenizer.word\_index 확인
- 단어 빈도 확인
- 빈도가 적은 단어 제거
- pad\_sequences()로 패딩 수행

### 출력 결과

- tokenizer 생성
- fit\_on\_texts()
- texts\_to\_sequences()
- word\_to\_index 출력
- vocab\_size 확인
- 단어 집합 크기 확인
- 문장 길이 확인
- max\_len 계산
- pad\_sequences()
- 학습 데이터 Shape 출력

### CNN 모델 생성

- 필요한 클래스 import
- 모델 생성

## 모델 구성

- Embedding
- Dropout(30%)
- Conv1D
- Kernel Size 지정
- 마지막 활성화함수  
이진 분류 Sigmoid  
다중 분류 Softmax
- EarlyStopping
- ModelCheckpoint
- evaluate()

## 출력 결과

### 모델 생성 코드

- Embedding
- Dropout
- Conv1D
- GlobalMaxPooling1D
- Dense
- compile()

### 모델 학습

`model.fit()`

### 모델 평가

`model.evaluate()`

## 결과

CNN을 이용한 문장 분류의 정확도가 높음을 확인

## CNN 스팸 메일 분류기

### 실습 순서

1. 데이터 불러오기
2. 불필요한 열 삭제
3. 문자 → 숫자 변환
4. 중복 제거
5. value\_counts()
6. train\_test\_split()
7. tokenizer 생성
8. X\_train\_encoded 출력
9. 희귀 단어 제거
10. pad\_sequences()

### CNN 모델 만들기

1. 필요한 클래스 import
2. 모델 설계
  - Embedding
  - Dropout
  - 필터(Filter) 개수 지정
  - Kernel Size 지정
3. 모델 생성
4. EarlyStopping
5. ModelCheckpoint
6. evaluate()를 이용한 성능 평가

### 3. 자연어 처리를 위한 RNN

#### 1. RNN 데이터 입력을 위한 데이터 로더 구현

##### 1) Field 지정

###### (1) torchtext 를 이용한 Field 객체 생성

- text field 객체 및 label field 객체 생성

예시 데이터

```
['Pushing', 'Daisies', 'truly', 'is', 'a', 'landmark', ...]
```

레이블

```
pos
```

구성

- text : IMDB 데이터 세트 영화 리뷰
- label : IMDB 데이터 세트 긍정·부정

예시 코드

```
from torchtext.legacy.data import Field

text_field = Field(
    sequential=True,
    include_lengths=True,
    fix_length=200
)

label_field = Field(
    sequential=False
)
```



## 2) 데이터 세트 생성

### (1) torchtext 를 이용한 IMDB 데이터 세트 로드

#### IMDB 데이터 세트 로드

- ① text\_field 와 label\_field 에 할당
- ② split() 함수를 이용하여 train, test 데이터 세트 분리

```
from torchtext.legacy.datasets import IMDB

train, test = IMDB.splits(
    text_field,
    label_field
)
```

#### train 데이터 출력

- ① 첫 번째 데이터 출력
- ② 첫 번째 레이블 출력

```
print(vars(train.examples[0]))
print(vars(train.examples[0])['label'])
```

#### 출력 예

```
{
  'text':
  ['Pushing', 'Daisies', 'truly', 'is', 'a', 'landmark', ...]
}

pos
```

### 3) Vocabulary 생성

#### (1) text field 와 label field Vocabulary

##### build\_vocab() 함수 이용

```
text_field.build_vocab(  
    train,  
    vectors='fasttext.simple.300d'  
)  
  
label_field.build_vocab(train)
```

##### Pre-trained 임베딩 모델 사용

```
vectors='fasttext.simple.300d'
```

### 3) Vocabulary 생성

#### 지원되는 Pre-trained 임베딩 모델

```
charngram.100d  
fasttext.en.300d  
fasttext.simple.300d  
glove.42B.300d  
glove.840B.300d  
glove.twitter.27B.25d  
glove.twitter.27B.50d  
glove.twitter.27B.100d  
glove.twitter.27B.200d  
glove.6B.50d  
glove.6B.100d  
glove.6B.200d  
glove.6B.300d
```

##### 사용자 정의 임베딩 모델

```
torchtext.vocab.Vocab.set_vectors()
```

함수를 이용하여 사용할 수 있다.

## 4) 데이터 로더 구현

### (1) BucketIterator 클래스를 이용한 데이터 로더 구현

train 과 test 데이터 로더 생성

```
import torch
from torchtext.legacy.data import BucketIterator

device = 'cuda' if torch.cuda.is_available() else 'cpu'
batch_size = 32

train_iter, test_iter = BucketIterator.splits(
    (train, test),
    batch_size=batch_size,
    device=device
)
```

## 2. LSTM / GRU 모델 활용

### 1) PyTorch Lightning 개요

#### (1) PyTorch Lightning 정의

PyTorch 의 High Level 인터페이스를 제공하는 오픈소스 Python 라이브러리

#### PyTorch 와 PyTorch Lightning 관계

TensorFlow 와 Keras 의 관계와 유사

#### PyTorch Lightning

PyTorch 의 Training, Testing 을 보다 간결한 코드로 사용할 수 있도록 구현된 프레임워크

## PyTorch Lightning 정의

### 기존 PyTorch

```
num_epochs = 1

for epoch in range(num_epochs):
    for train_batch in mnist_train:
        x, y = train_batch
        logits = pytorch_model(x)
        loss = cross_entropy_loss(logits, y)
        print(loss.item())
        loss.backward()
```

## PyTorch Lightning

```
model = LightningMNISTClassifier()
trainer = pl.Trainer()
trainer.fit(model)
```

## PyTorch Lightning 설치

```
pip install pytorch-lightning
```

torchtext 설치

```
pip install torchtext
```

## 2) PyTorch Lightning 을 이용한 LSTM 구현

### (1) LightningModule 클래스 상속

```
import pytorch_lightning as pl

class RNNModel(pl.LightningModule):
```

## (2) init 함수 재정의

생성자 구현 및 주요 변수 초기화

```
def __init__(self,
                embedding,
                lstm_input_size=300,
                lstm_hidden_size=...):
    super().__init__()
    self.embedding = embedding
    self.lstm = nn.LSTM(...)
    self.lin = nn.Linear(...)
    self.loss_function = nn.CrossEntropyLoss()
    self.train_accuracy = pl.metrics.Accuracy()
    self.val_accuracy = pl.metrics.Accuracy()
```

## 3) forward 함수 재정의

forward() 함수로 신경망 구현

```
def forward(self, X):
    x = self.embedding[X]
    x = x.to(self.device)
    x = x.permute(1,0,2)
    x, _ = self.lstm(x)
    x = F.elu(x.permute(1,0,2))
    x = self.lin(x)
    x = x.sum(dim=1)
    return x
```

#### 4) Training / Validation / Test 함수 재정의

##### 학습 단계별 Step 함수 구현

```
def training_step(...):  
    ...  
    loss = self.loss_function(...)  
    train_acc = ...  
    self.log(...)  
    return ...  
  
def validation_step(...):  
    ...  
    val_acc = ...  
    self.log(...)  
    return ...
```

##### 데이터 로더 함수

```
def train_dataloader(self):  
    return train_iter  
  
def val_dataloader(self):  
    return test_iter
```

##### Optimizer 설정

```
def configure_optimizers(self):  
    return Adam(  
        self.parameters(),  
        lr=0.01  
    )
```

### 3) 모델 객체 생성 및 학습

#### 사용자 정의 모델 생성

```
model = RNNModel(  
    text_field.vocab.vectors  
)
```

#### Trainer 생성

```
trainer = pl.Trainer(  
    gpus=1,  
    max_epochs=3  
)
```

#### 학습

```
trainer.fit(model)
```

#### 학습 수행 결과 평가

##### 평가 지표

- train accuracy
- validation accuracy

##### 학습 결과 예시

1564/1564

loss = 0.396

train\_acc = ...

학습이 완료된 후 **Train Accuracy** 와 **Validation Accuracy** 를 이용하여 모델의 성능을 평가한다.